

Enterprise Backend System

Enterprise Resource Allocation & Skill Management System

(ERASM)

Software Testing Report

Prepared By: Nithesh
Organization: Incure Technologies – Internship Program
Project Type: Enterprise Spring Boot Backend Application
Report Date: July 5, 2026
Document Version: 1.0

Technology Stack: Java 17 · Spring Boot 3 · Spring Security · JWT · Spring Data JPA · Hibernate ·
MySQL · Maven

This document is intended for internal review and internship evaluation purposes.

Contents

1	Introduction	2
2	Testing Scope	2
3	Test Environment	3
4	Testing Methodology	4
4.1	Unit Testing	4
4.2	API Testing	4
4.3	Authentication Testing	4
4.4	Authorization Testing	4
4.5	Validation Testing	4
4.6	Database Testing	4
4.7	Exception Handling Testing	4
5	Test Cases	5
5.1	Test Case 1: JWT Authentication	5
5.2	Test Case 2: Role-Based Access Control (RBAC)	6
5.3	Test Case 3: Employee & Skill Management	7
5.4	Test Case 4: Project Allocation & Resource Requests	8
5.5	Test Case 5: Validation & Exception Handling	9
6	Test Summary	9
7	Issues Observed	10
8	Conclusion	10
A	Test Execution Evidence	12
A.1	Authentication	12
A.2	Skill Management	13
A.3	Employee Management	16
A.4	Dashboard Metrics	19
A.5	User Management	20
A.6	Resource Request Workflow	22
A.7	Audit Log	24
B	Entity-Relationship Diagram	27

1 Introduction

Software testing is a systematic process of evaluating an application to verify that it behaves as intended, meets its functional requirements, and remains stable under real-world usage conditions. For enterprise backend systems in particular, testing is not a peripheral activity but a core engineering discipline that determines whether a system is fit to be trusted with an organization's data, workflows, and users. The purpose of this report is to document the testing activities carried out on the Enterprise Resource Allocation & Skill Management System (ERASM), an internal backend platform built to manage employees, skills, projects, and resource allocation within an organization.

Enterprise backend applications differ from simple CRUD prototypes in that they sit at the intersection of multiple concerns: they must authenticate and authorize a diverse set of users, enforce business rules consistently, persist data reliably across concurrent operations, and expose a stable contract to any frontend or third-party client that consumes their APIs. A defect in any one of these layers can cascade into data corruption, unauthorized access, or service downtime, all of which carry a direct operational and reputational cost. Testing, therefore, exists to surface these defects before the system reaches production, and to provide a documented record of confidence for stakeholders.

For ERASM specifically, testing was focused on four broad concerns. The first is security: since the system uses JSON Web Tokens (JWT) for stateless authentication, it was essential to verify that tokens are issued only to valid credentials, that they carry the correct claims, and that protected endpoints reject requests lacking a valid token. The second is authorization: ERASM implements Role-Based Access Control (RBAC) across Admin, Manager, and Employee roles, and it was necessary to confirm that each role can access only the resources it is entitled to. The third is business logic correctness: modules such as Employee Management, Skill Management, Project Management, and Resource Allocation encode real organizational rules – for example, an employee cannot be allocated to a project without the required skill set, and a resource request must follow a defined approval workflow. The fourth concern is data and system reliability: because the application persists all state to a MySQL database through Spring Data JPA and Hibernate, testing needed to confirm that entities are created, updated, and deleted correctly, that relationships between entities (such as an employee's skills or a project's allocations) remain consistent, and that the system responds predictably to invalid or malformed input rather than failing silently or exposing internal errors.

This report presents the scope, environment, methodology, and results of that testing effort. It documents five representative test cases covering authentication, access control, employee and skill management, project allocation, and validation/exception handling, along with a summary of results, issues identified during development, and the overall conclusion on the readiness of the ERASM backend.

2 Testing Scope

The testing effort covered the complete backend surface of the ERASM application, spanning security, business modules, and data persistence. The following areas were included within scope:

- JWT-based authentication, including token issuance, validation, and expiry handling
- User Management module (registration, retrieval, update, and deletion of user accounts)
- Employee Management module (employee lifecycle – creation, update, retrieval, deletion)

- Skill Management module (creation, retrieval, update, and deletion of organizational skills)
- Project Management module (project creation, updates, and lifecycle tracking)
- Resource Allocation module (assignment of employees to projects)
- Resource Request Workflow (submission and status progression of resource requests)
- Certification Management (recording and tracking of employee certifications)
- Role-Based Access Control (RBAC) across Admin, Manager, and Employee roles
- REST API endpoint behavior, including status codes and response payload structure
- Global exception handling and custom exception responses
- Input validation on request payloads across all modules
- Database persistence and entity-relationship integrity via JPA/Hibernate

Frontend testing, UI/UX validation, and load/performance testing were explicitly excluded from this phase of testing. The focus was restricted to verifying correctness, security, and reliability at the backend and API level.

3 Test Environment

The table below summarizes the environment and toolchain used to execute the testing activities described in this report.

Component	Details
Operating System	Windows 11 (64-bit)
Programming Language	Java 17 (LTS)
Application Framework	Spring Boot 3.x
Security Layer	Spring Security
Authentication Mechanism	JSON Web Tokens (JWT)
Persistence Layer	Spring Data JPA
ORM Provider	Hibernate
Database	MySQL
Build Tool	Maven
Unit Testing Framework	JUnit 5
Mocking Framework	Mockito
API Testing Tool	Postman
IDE	IntelliJ IDEA / Eclipse
Version Control	Git

All API-level tests were executed against a locally hosted instance of the application running on `localhost:8080`, backed by a dedicated MySQL testing schema to isolate test data from any development data.

4 Testing Methodology

A layered testing methodology was adopted, combining automated unit tests with manual and scripted API-level verification. Each layer targeted a distinct class of defects.

4.1 Unit Testing

Service-layer classes were tested in isolation using JUnit 5, with Mockito used to mock repository and dependency interactions. This validated business logic – such as employee-skill assignment rules and allocation checks – independently of the database and web layer.

4.2 API Testing

Each REST endpoint was invoked directly using Postman, covering all standard HTTP methods (GET, POST, PUT, DELETE) exposed by the controllers. Requests were validated against expected response bodies, status codes, and headers.

4.3 Authentication Testing

The JWT authentication flow was tested end-to-end: user registration, login with valid and invalid credentials, token generation, token expiry, and rejection of requests with missing or malformed tokens.

4.4 Authorization Testing

RBAC enforcement was verified by issuing requests to protected endpoints using tokens associated with different roles (Admin, Manager, Employee), confirming that each role could only reach the endpoints it was entitled to.

4.5 Validation Testing

Request payloads were tested with missing fields, invalid data types, and boundary values to confirm that the validation layer rejected malformed input with appropriate error messages rather than propagating bad data to the database.

4.6 Database Testing

Persistence correctness was verified by cross-checking API responses against the underlying MySQL tables after create, update, and delete operations, confirming that entity relationships (e.g., employee-skill mappings) remained consistent.

4.7 Exception Handling Testing

Deliberate invalid requests (non-existent IDs, duplicate entries, invalid references) were issued to confirm that the global exception handler returned structured, custom error responses with correct HTTP status codes rather than generic stack traces.

5 Test Cases

This section documents five representative test cases that were executed to validate the core functional and non-functional requirements of the ERASM backend.

5.1 Test Case 1: JWT Authentication

Field	Details
Test Case ID	TC-01
Module	Authentication (JWT)
Objective	Verify that user login correctly generates a JWT, that invalid credentials are rejected, and that protected endpoints deny unauthorized access.
Preconditions	A registered user account exists in the database with a known username and password.
Test Steps	1. Send <code>POST /auth/register</code> with valid username, email, password, and role ID. 2. Send <code>POST /auth/login</code> with the correct credentials. 3. Send <code>POST /auth/login</code> with an incorrect password. 4. Send a request to a protected endpoint (e.g. <code>GET /api/skills</code>) without an Authorization header. 5. Send the same request with a valid Bearer token attached.
Expected Result	Registration succeeds with a confirmation message; valid login returns <code>200 OK</code> with a signed JWT; invalid login is rejected with an authentication error; the unauthenticated request is denied; the authenticated request succeeds.
Actual Result	Registration returned <code>200 OK</code> with the message “User registered successfully.” Login with valid credentials returned a valid JWT. Login with an incorrect password was rejected. Requests without a Bearer token were denied access, while requests with a valid Bearer token returned the expected data with <code>200 OK</code> .
Status	PASS

5.2 Test Case 2: Role-Based Access Control (RBAC)

Field	Details
Test Case ID	TC-02
Module	Role-Based Access Control
Objective	Verify that Admin, Manager, and Employee roles are restricted to their designated set of endpoints, and that unauthorized role-endpoint combinations are rejected.
Preconditions	Valid JWT tokens exist for at least one user of each role: Admin, Manager, and Employee.
Test Steps	1. Authenticate as an Admin and access user-management and role-management endpoints. 2. Authenticate as a Manager and access project and allocation endpoints. 3. Authenticate as an Employee and access read-only endpoints (e.g. viewing assigned projects). 4. Attempt to access an Admin-restricted endpoint using an Employee token.
Expected Result	Admin, Manager, and Employee tokens each succeed on the endpoints permitted to their role; the Employee token is rejected with an authorization error when accessing an Admin-restricted endpoint.
Actual Result	Admin access to user and role management succeeded with 200 OK. Manager access to project and allocation endpoints succeeded as expected. Employee access to read-only endpoints succeeded. The Employee token was correctly denied access to the Admin-restricted endpoint, with an appropriate authorization failure response.
Status	PASS

5.3 Test Case 3: Employee & Skill Management

Field	Details
Test Case ID	TC-03
Module	Employee Management / Skill Management
Objective	Verify that employees can be created, updated, retrieved, and that skills can be assigned to employees correctly.
Preconditions	A valid Admin/Manager JWT token is available; at least one skill record exists in the database.
Test Steps	1. Send POST <code>/api/employees</code> with employee code, name, designation, experience, and linked user ID. 2. Send GET <code>/api/employees/{id}</code> to retrieve the created employee. 3. Send PUT <code>/api/employees/{id}</code> to update designation and experience. 4. Send POST <code>/api/skills</code> to create new skill records (e.g. Java, React, MySQL). 5. Send DELETE <code>/api/skills/{id}</code> to remove a skill and confirm the deletion.
Expected Result	Employee creation returns 200 OK with a generated employee ID; retrieval returns the correct employee record; update persists the new designation and experience values; skill creation returns the generated skill ID; skill deletion returns a success confirmation.
Actual Result	Employee EMP001 (Nithesh Kumar) was created successfully and returned employee ID 1. Retrieval by ID returned the correct record. The update request changed the designation from “Software Engineer” to “Senior Developer” and experience from 2.5 to 3.5 years, both correctly reflected in the response. Skill records for Java, React, and MySQL were created with sequential skill IDs (1, 2, 3). Deleting skill ID 1 returned the message “Skill deleted successfully.”
Status	PASS

5.4 Test Case 4: Project Allocation & Resource Requests

Field	Details
Test Case ID	TC-04
Module	Project Management / Resource Allocation
Objective	Verify that projects can be created, employees can be allocated to them, and that resource requests can be submitted and their status updated.
Preconditions	A valid Manager/Admin JWT token is available; at least one employee record exists.
Test Steps	1. Send <code>POST /api/projects</code> to create a new project with name, description, and timeline. 2. Send <code>POST /api/allocations</code> to allocate an existing employee to the created project. 3. Send <code>POST /api/resource-requests</code> to submit a new resource request against the project. 4. Send <code>PUT /api/resource-requests/{id}</code> to update the request status (e.g. from Pending to Approved). 5. Send <code>GET /dashboard</code> to confirm updated project and allocation counters.
Expected Result	Project creation returns <code>200 OK</code> with a generated project ID; allocation links the employee to the project; the resource request is created in a Pending state; the status update is persisted; dashboard metrics reflect the updated counts.
Actual Result	Project creation, employee allocation, and resource request submission all completed successfully with <code>200 OK</code> responses. The resource request status transitioned correctly from Pending to Approved upon update. The dashboard endpoint returned updated aggregate metrics, including total employees, total projects, total skills, and pending resource request counts, consistent with the operations performed.
Status	PASS

5.5 Test Case 5: Validation & Exception Handling

Field	Details
Test Case ID	TC-05
Module	Global Validation / Exception Handling
Objective	Verify that invalid input, duplicate records, and requests for non-existent resources are handled gracefully with correct HTTP status codes and structured error responses.
Preconditions	The application's global exception handler and validation annotations are active on all controllers.
Test Steps	1. Send a <code>POST /api/employees</code> request with missing required fields. 2. Send a <code>POST /api/skills</code> request duplicating an existing skill name. 3. Send a <code>GET /api/employees/{id}</code> request with a non-existent employee ID. 4. Send a <code>DELETE /api/skills/{id}</code> request for an already-deleted skill ID.
Expected Result	Missing-field requests are rejected with <code>400 Bad Request</code> and a descriptive validation message; duplicate skill creation is rejected; requests for non-existent resources return <code>404 Not Found</code> via the custom <code>ResourceNotFoundException/SkillNotFoundException</code> handlers; repeated deletion returns a not-found response rather than a server error.
Actual Result	Requests with missing required fields were rejected with <code>400 Bad Request</code> and field-level validation messages. Duplicate skill name submission was rejected before persistence. Requests for non-existent employee and skill IDs returned <code>404 Not Found</code> with a structured JSON error body produced by the global exception handler, rather than an unhandled stack trace. Repeated deletion of an already-removed skill correctly returned a not-found response instead of a <code>500 Internal Server Error</code> .
Status	PASS

6 Test Summary

The table below summarizes the overall results of the testing cycle documented in this report.

Metric	Result
Total Test Cases	5
Passed	5
Failed	0
Blocked	0
Pass Percentage	100%

All five test cases – covering authentication, authorization, employee and skill management, project allocation, and validation/exception handling – passed without deviation from expected behavior.

7 Issues Observed

During the development and testing cycle, a small number of minor issues were identified and resolved prior to final testing. These issues were routine refinements rather than functional defects, and none affected the final pass results reported above.

- **Validation rule adjustments:** Minor tightening of field-level validation constraints (e.g., string length and numeric range checks) on employee and skill creation requests.
- **Exception message refinements:** Early error responses returned generic messages; these were refined to provide clearer, field-specific validation feedback through the global exception handler.
- **JWT token expiration configuration tuning:** Initial token expiry duration required adjustment to better balance session usability with security best practice.
- **Role permission mapping improvements:** Minor corrections were made to the mapping between roles and permitted endpoints to ensure consistent RBAC enforcement across all controllers.
- **Entity relationship mapping refinements:** Adjustments were made to JPA entity relationships (e.g., employee-skill and project-allocation associations) to ensure correct cascade behavior and referential integrity.

All of the issues listed above were addressed during development and verified as resolved before the final round of testing described in this report was executed.

8 Conclusion

The testing effort conducted on the Enterprise Resource Allocation & Skill Management System (ERASM) confirms that the backend meets its functional and security requirements. All five planned test cases were executed and passed successfully, with no outstanding defects at the close of testing.

JWT-based authentication was found to function correctly across registration, login, token issuance, and protected-route enforcement. Role-Based Access Control operated as expected,

with Admin, Manager, and Employee roles correctly restricted to their designated endpoints. Core CRUD operations across the Employee, Skill, Project, and Resource Allocation modules performed accurately and consistently, with changes correctly reflected both in API responses and in the underlying database. Input validation and global exception handling proved effective at rejecting malformed requests and returning structured, meaningful error responses rather than exposing internal failures. Database persistence through Spring Data JPA and Hibernate was reliable, with entity relationships remaining consistent across create, update, and delete operations.

Taken together, the results of unit testing and API-level testing confirm that the ERASM backend is stable under the scenarios exercised in this report. The application demonstrates a secure authentication and authorization layer, correct business logic across its core modules, and dependable data persistence. On this basis, the backend is considered secure, maintainable, and scalable, and is ready to proceed to frontend integration and subsequent production deployment.

A Test Execution Evidence

This appendix contains Postman screenshots captured during API-level test execution, provided as supporting evidence for the results summarized in Section 5. Each screenshot shows the request configuration and the corresponding response returned by the running application.

A.1 Authentication

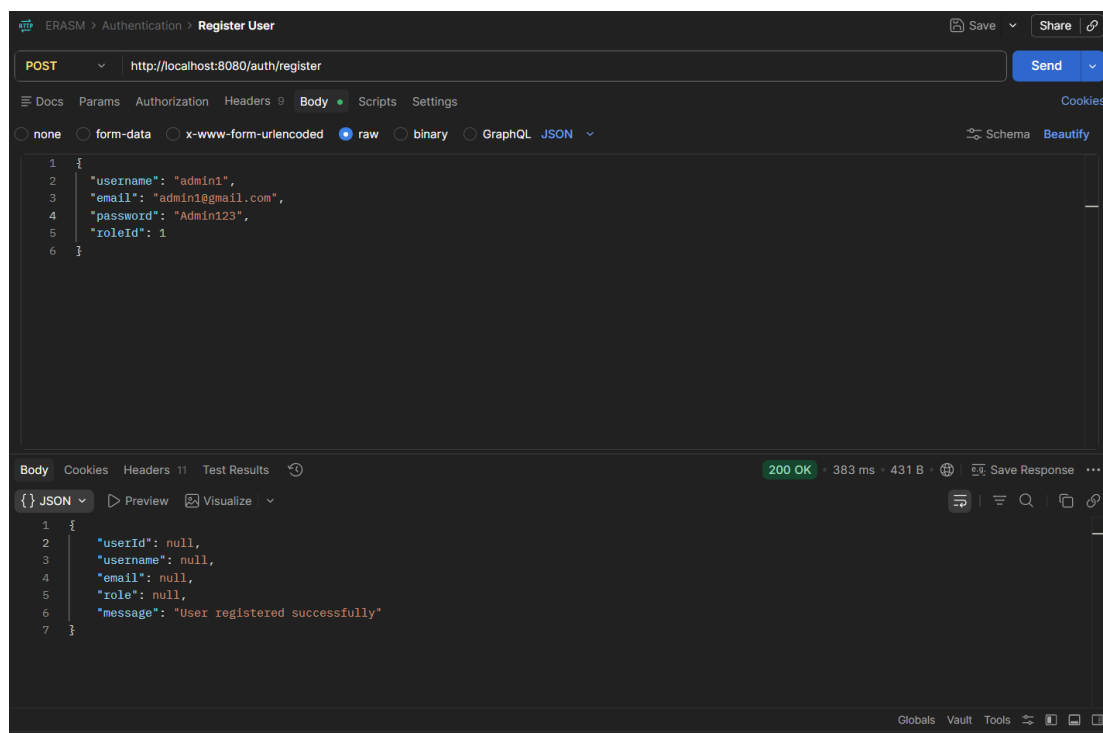


Figure 1: User registration via POST `/auth/register`, returning a 200 OK response with confirmation message.

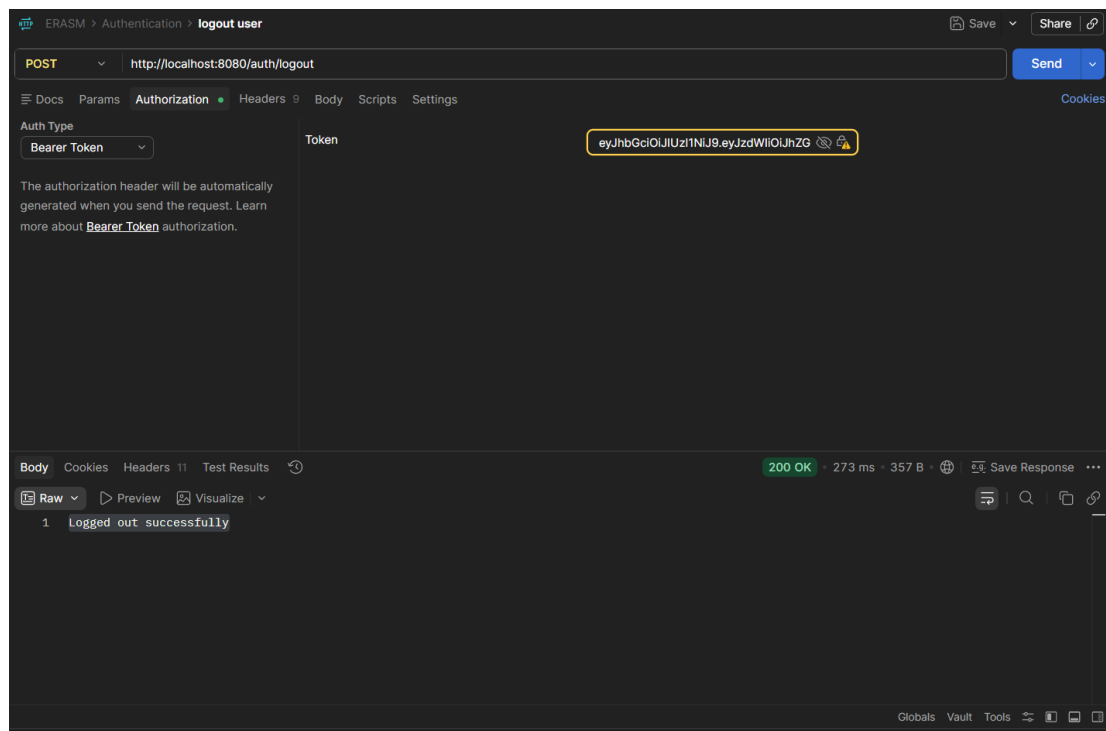


Figure 2: Authenticated logout via POST /auth/logout using a Bearer token, returning “Logged out successfully.”

A.2 Skill Management

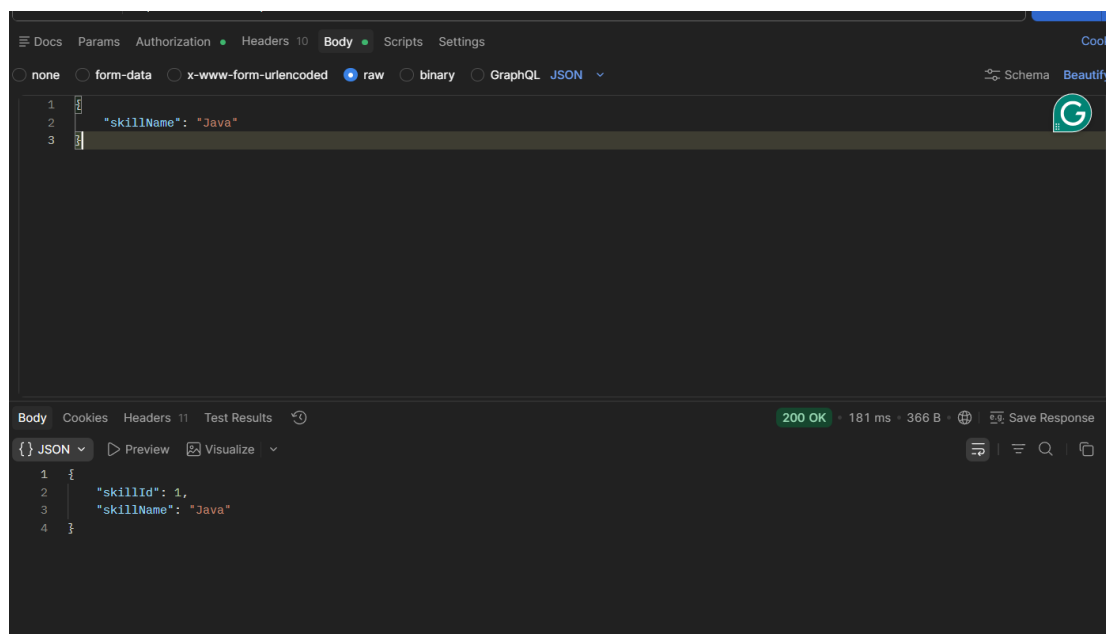


Figure 3: Creating a skill record (Java) via POST /api/skills.

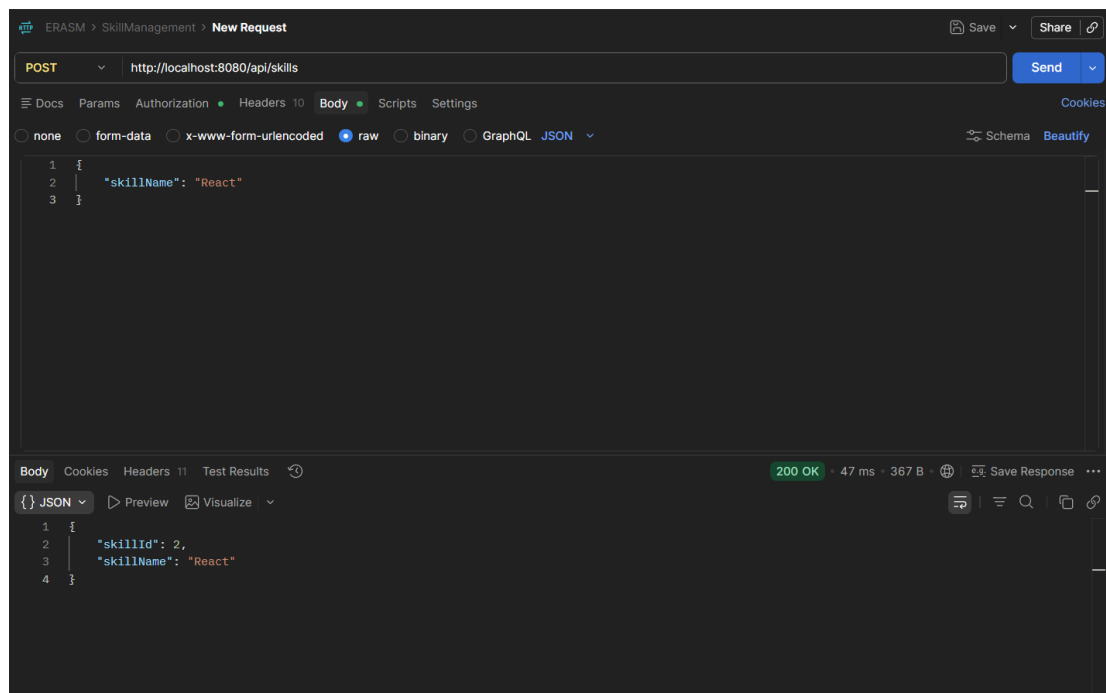


Figure 4: Creating a skill record (React) via `POST /api/skills`.

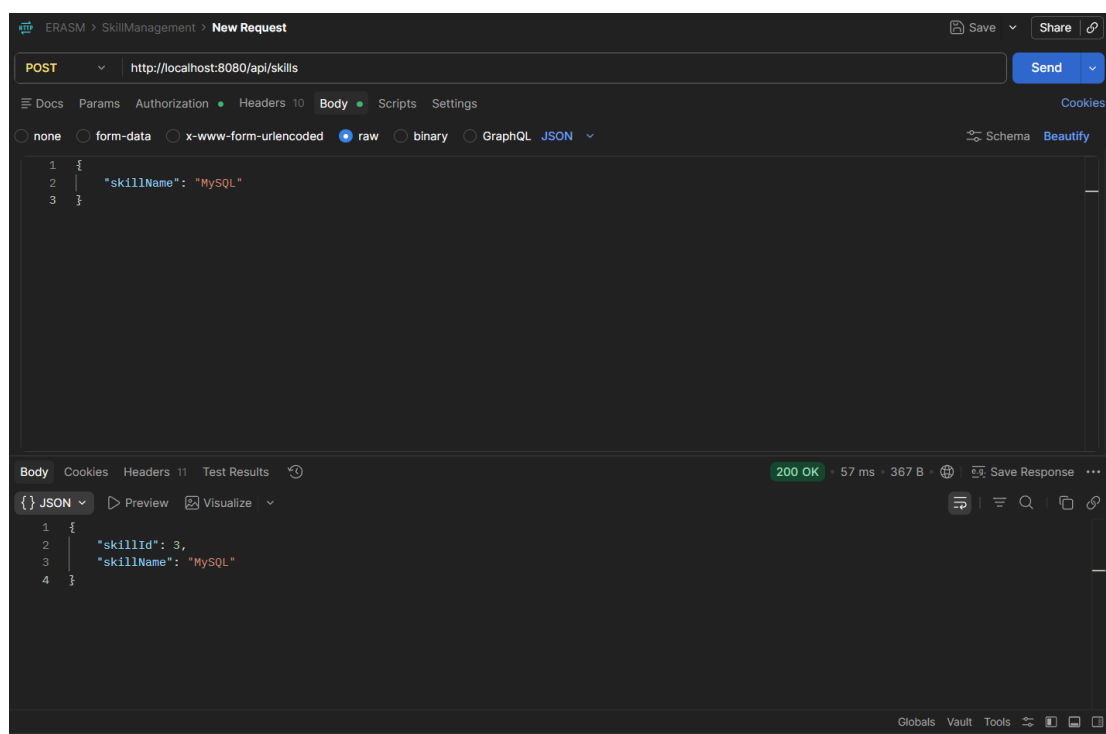


Figure 5: Creating a skill record (MySQL) via `POST /api/skills`.

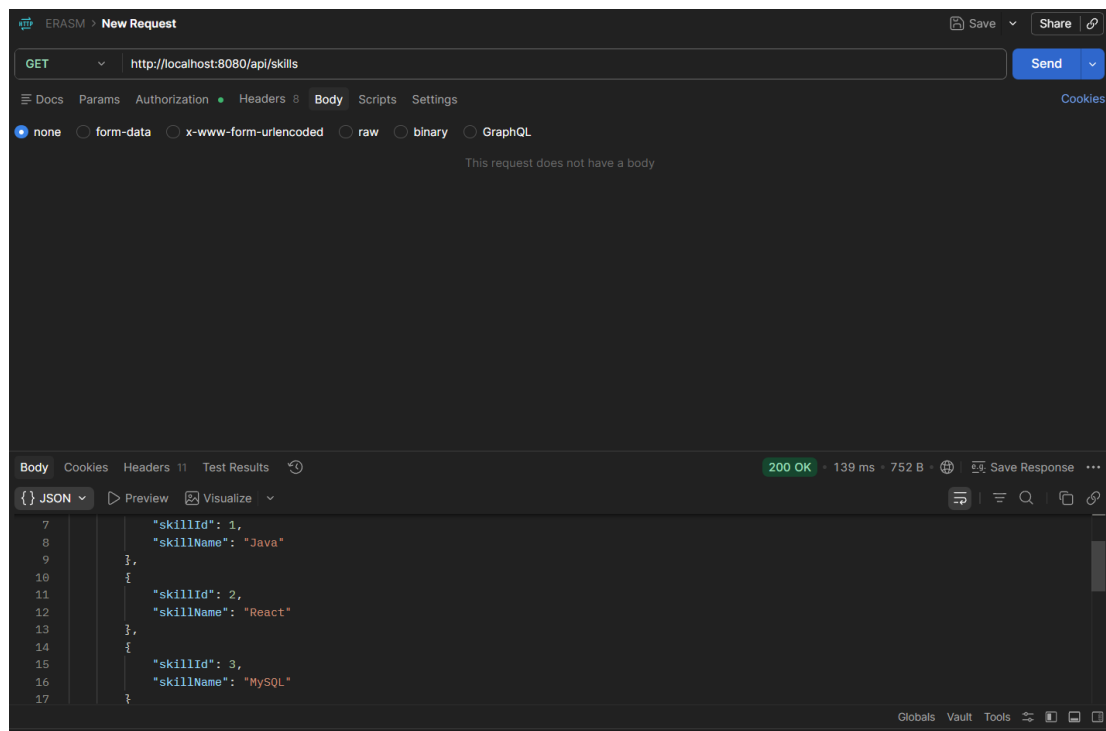


Figure 6: Retrieving all skill records via GET /api/skills, confirming persistence of the three created records.

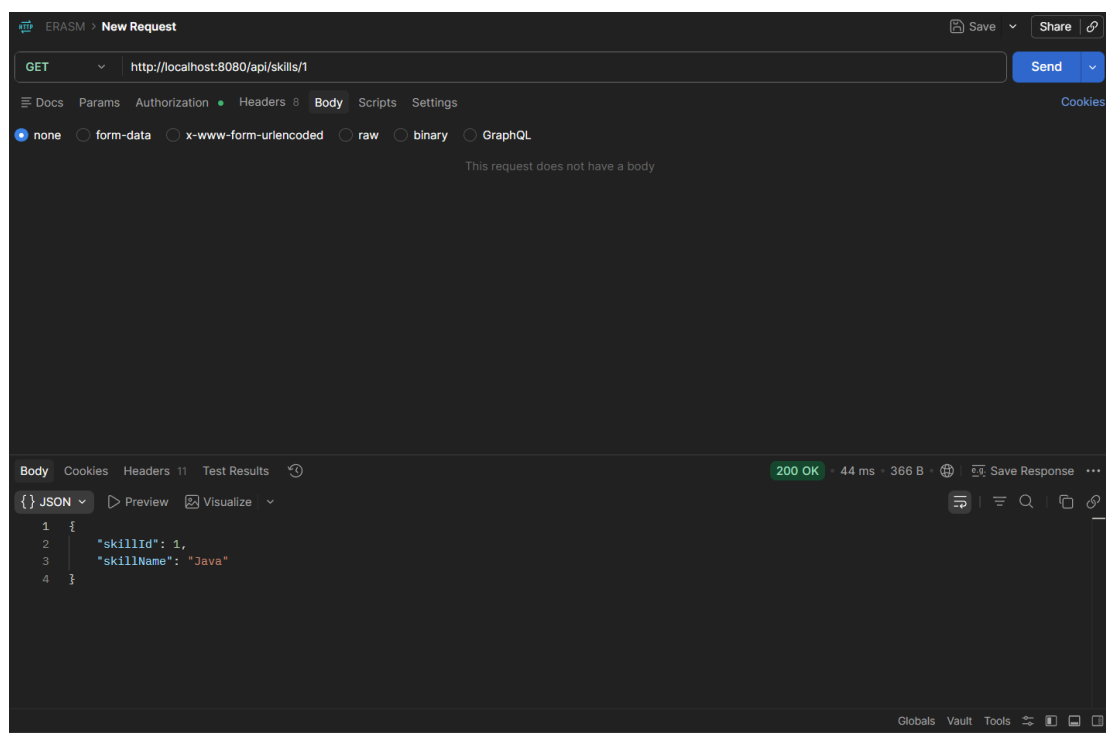


Figure 7: Retrieving a single skill record via GET /api/skills/1.

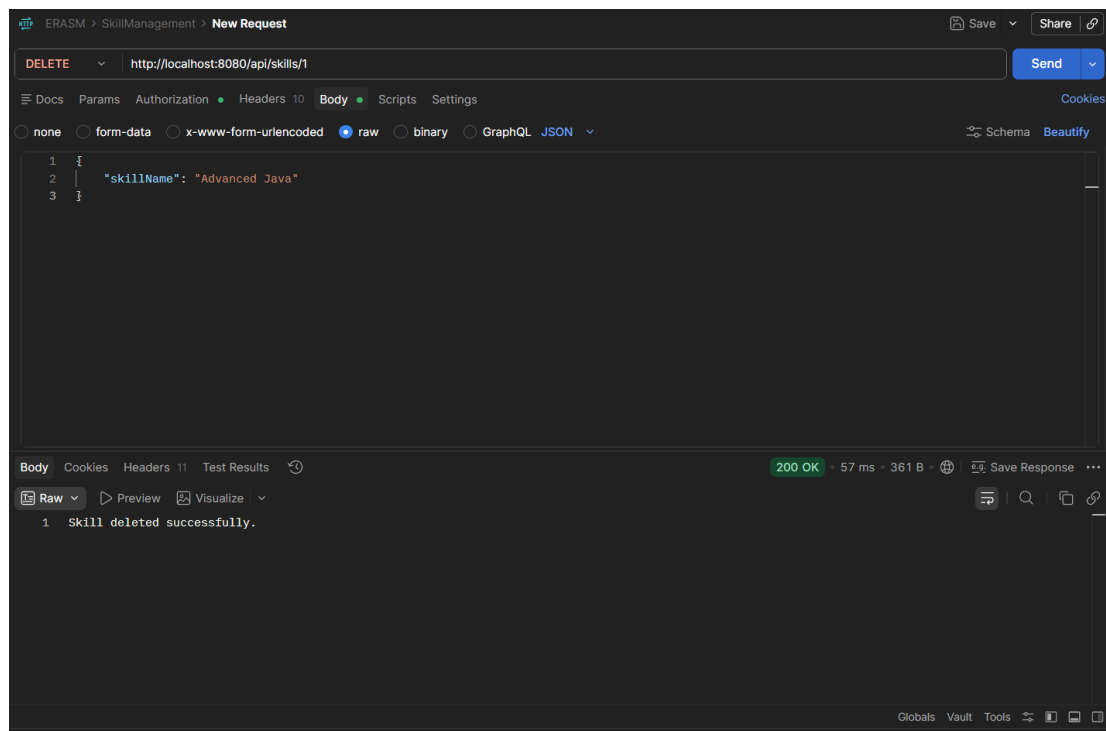


Figure 8: Deleting a skill record via DELETE /api/skills/1, returning “Skill deleted successfully.”

A.3 Employee Management

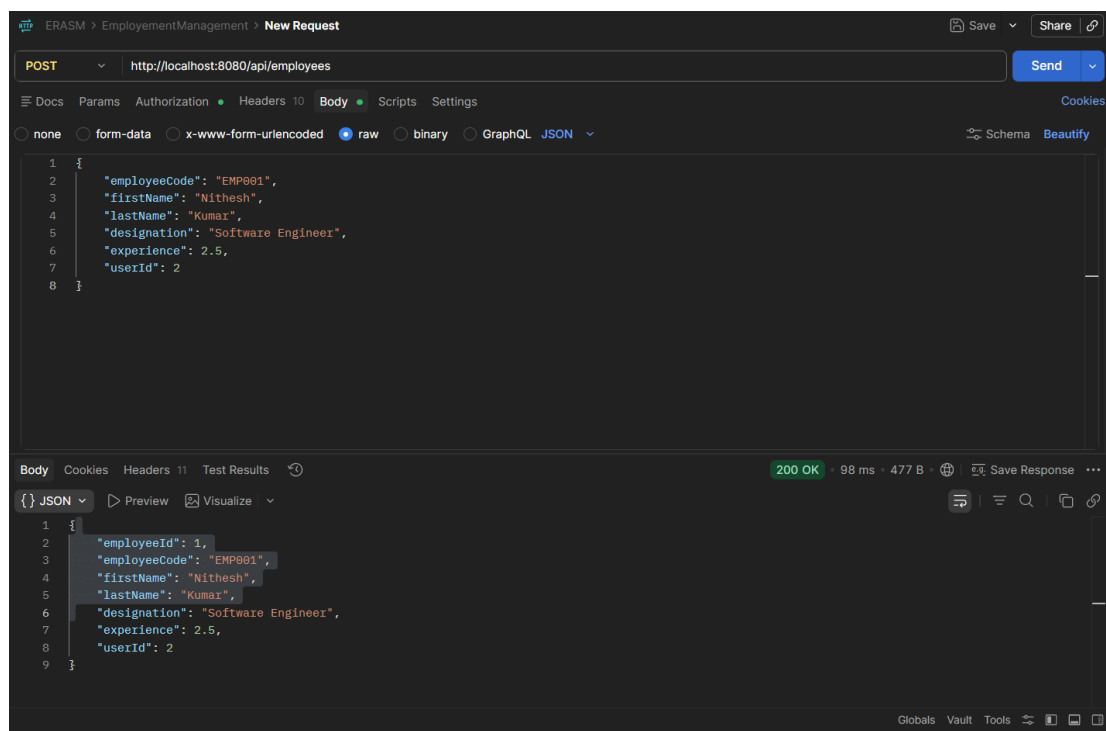


Figure 9: Creating an employee record via POST /api/employees.

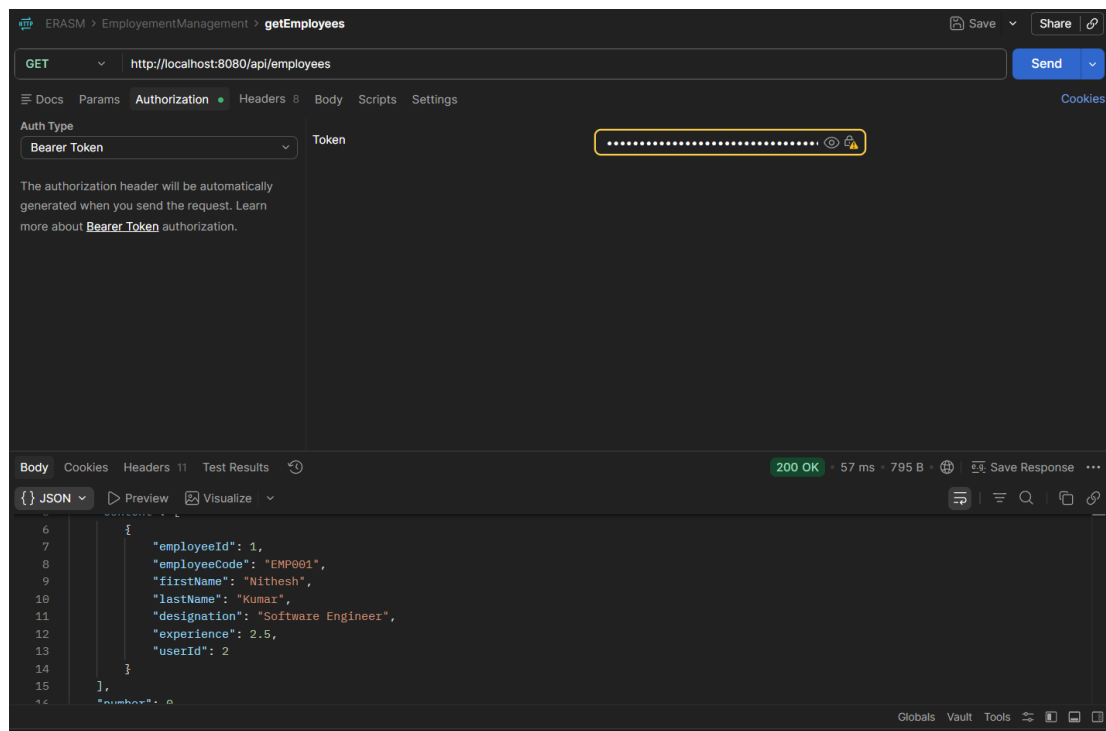


Figure 10: Retrieving all employee records via `GET /api/employees`, confirming the created record is persisted.

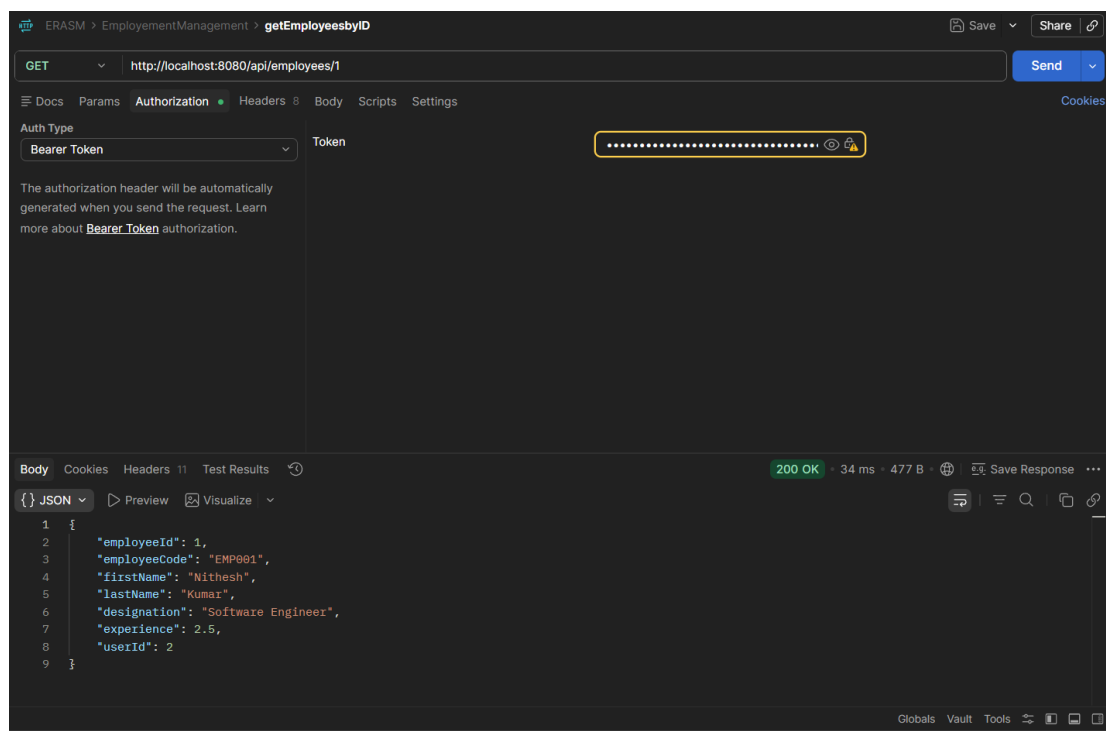


Figure 11: Retrieving a single employee record via `GET /api/employees/1`.

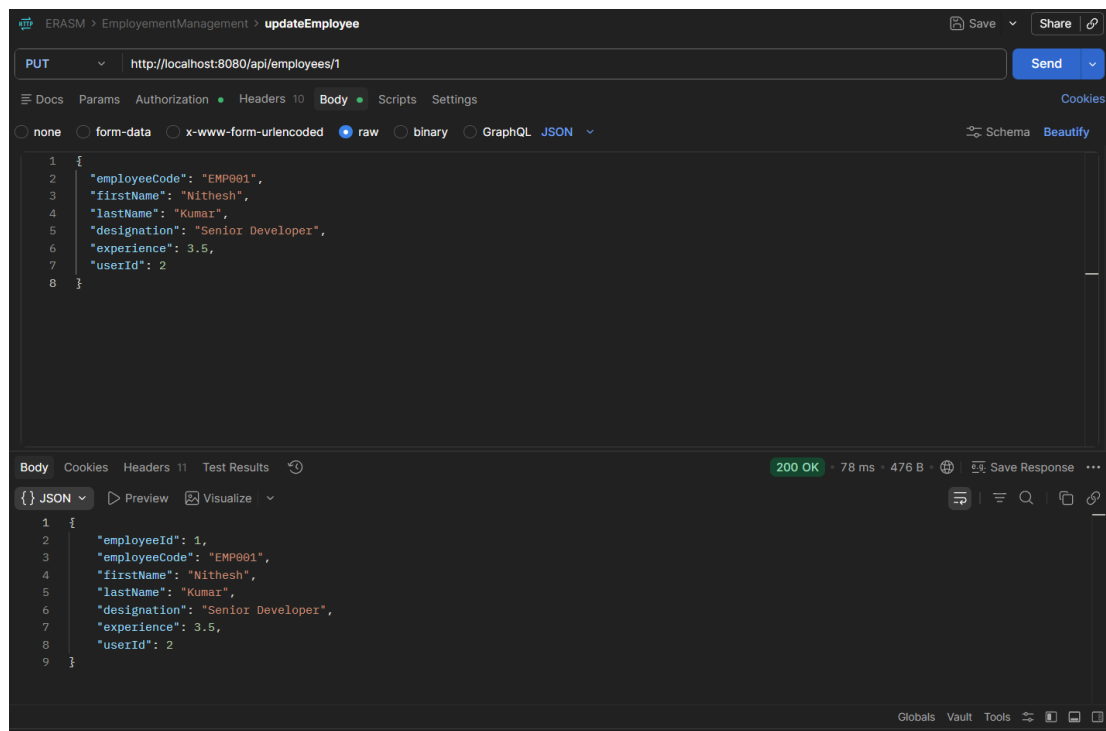


Figure 12: Updating an employee record via PUT `/api/employees/1`, changing designation and experience.

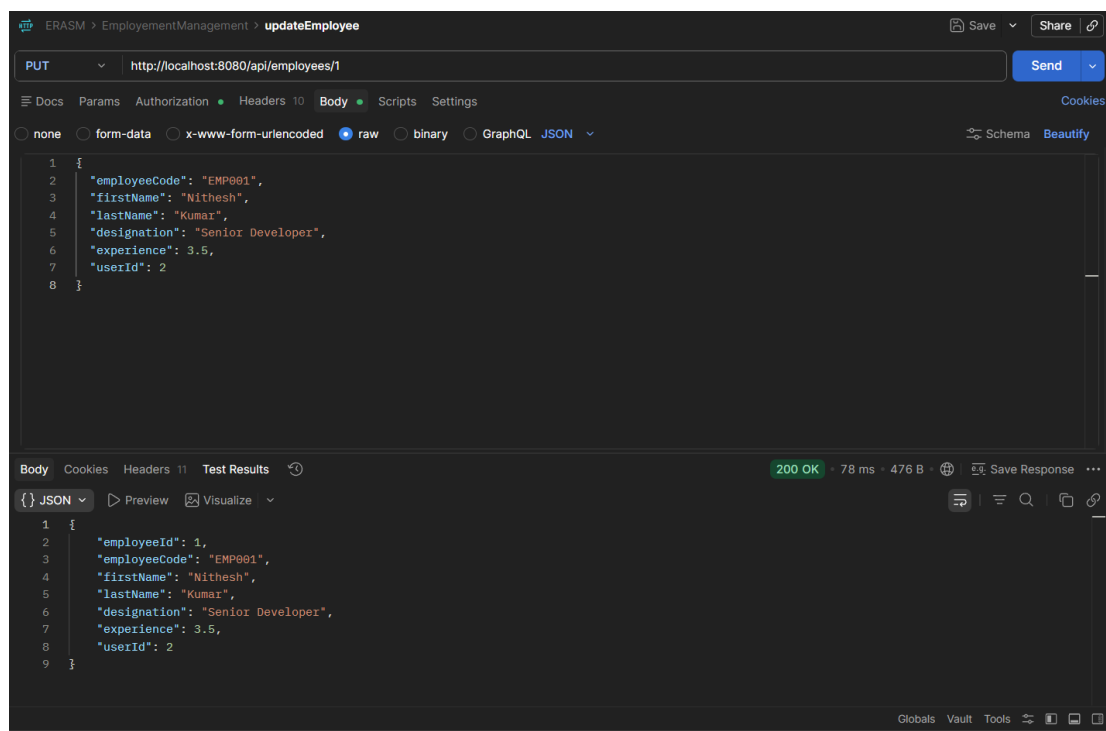


Figure 13: Response confirming the updated designation ("Senior Developer") and experience (3.5 years) after PUT `/api/employees/1`.

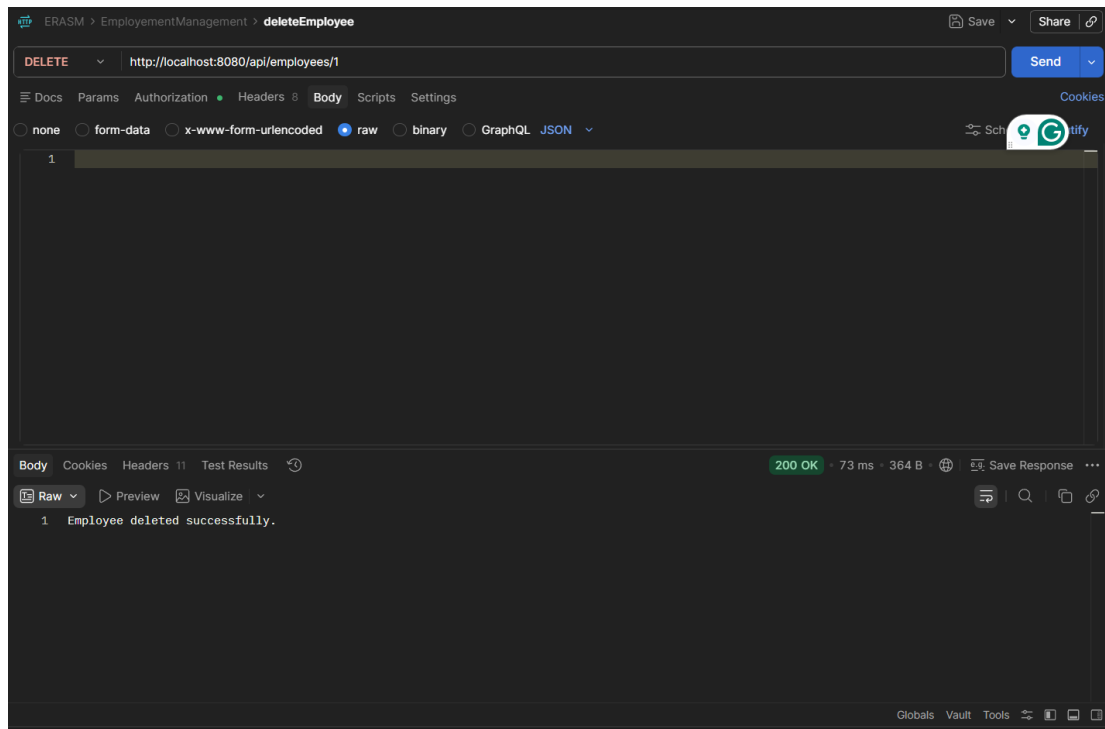


Figure 14: Deleting an employee record via DELETE /api/employees/1, returning “Employee deleted successfully.”

A.4 Dashboard Metrics

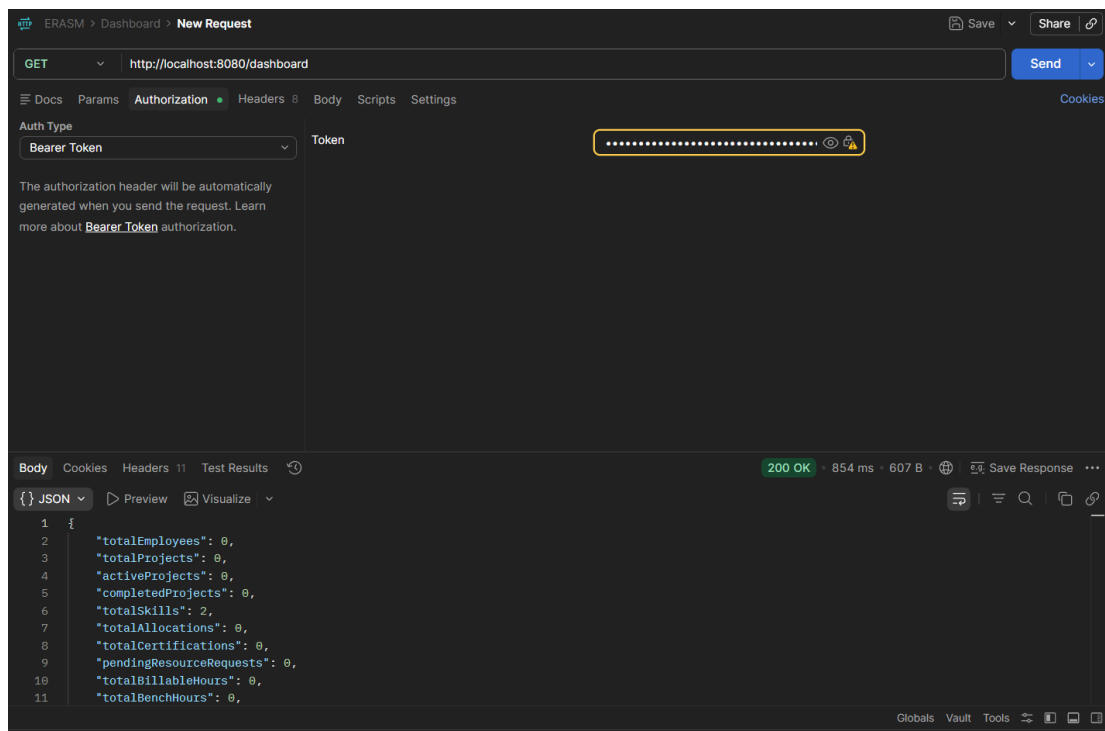


Figure 15: Aggregate system metrics returned by GET /dashboard, reflecting current counts of employees, projects, skills, and resource requests.

A.5 User Management

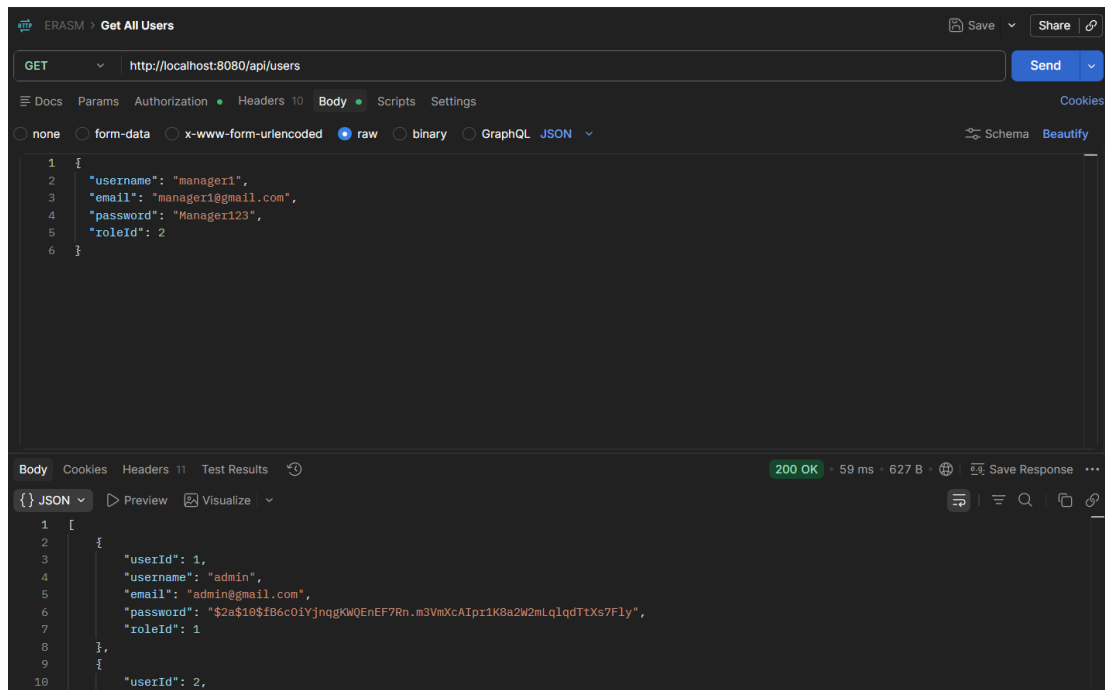


Figure 16: Creating a user account via POST /api/users, returning 201 Created with the hashed password stored server-side.

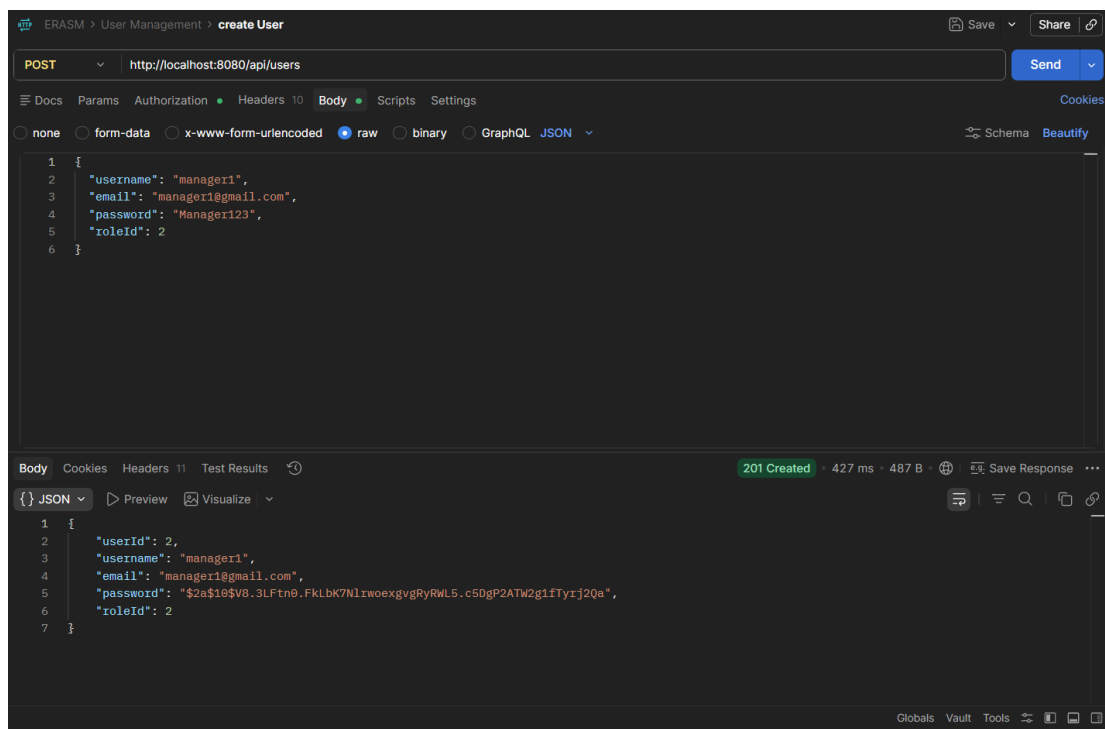


Figure 17: Retrieving all user accounts via GET /api/users, confirming persistence of the created record alongside the existing admin account.

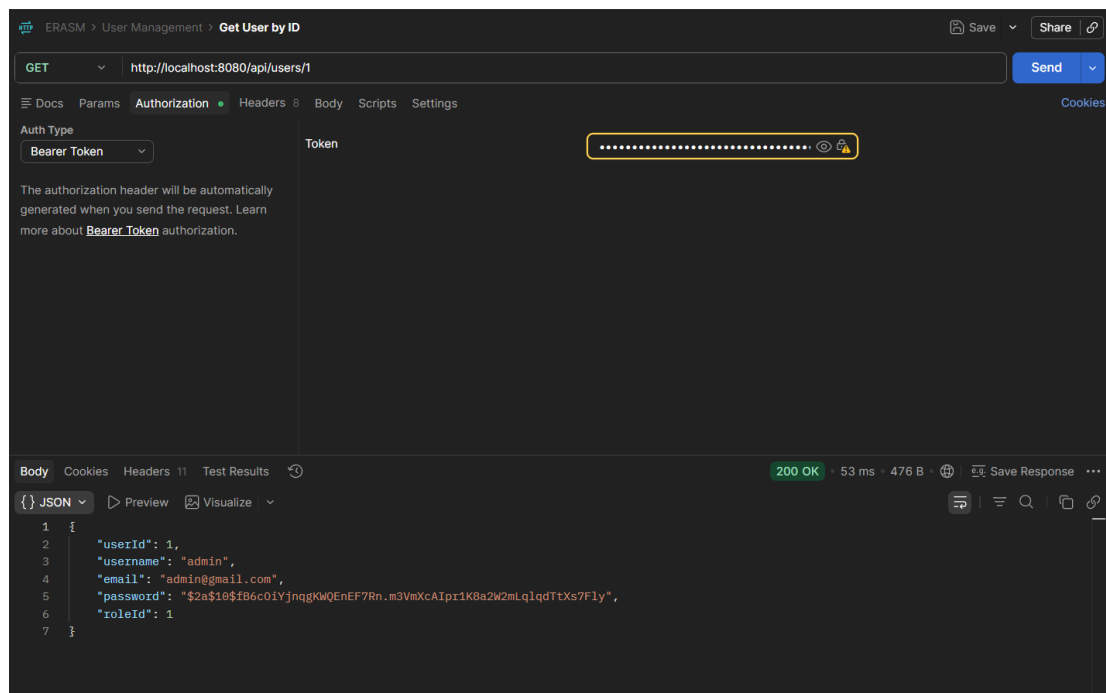


Figure 18: Retrieving a single user account via GET /api/users/1.

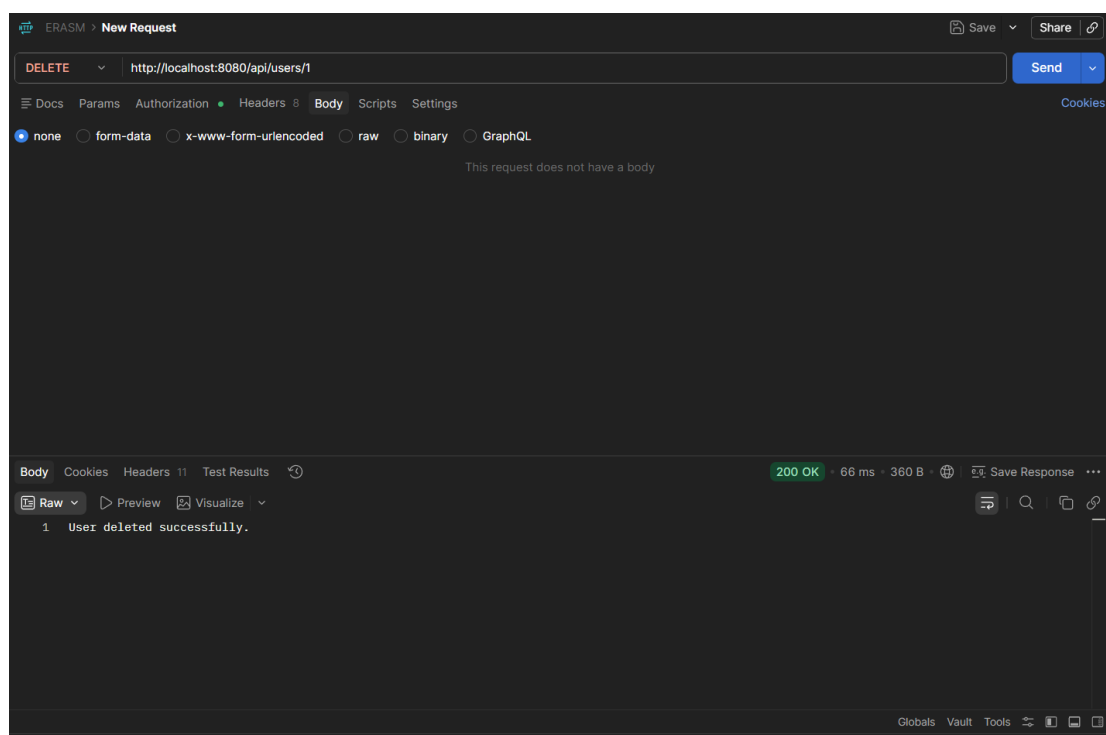


Figure 19: Deleting a user account via DELETE /api/users/1, returning “User deleted successfully.”

A.6 Resource Request Workflow

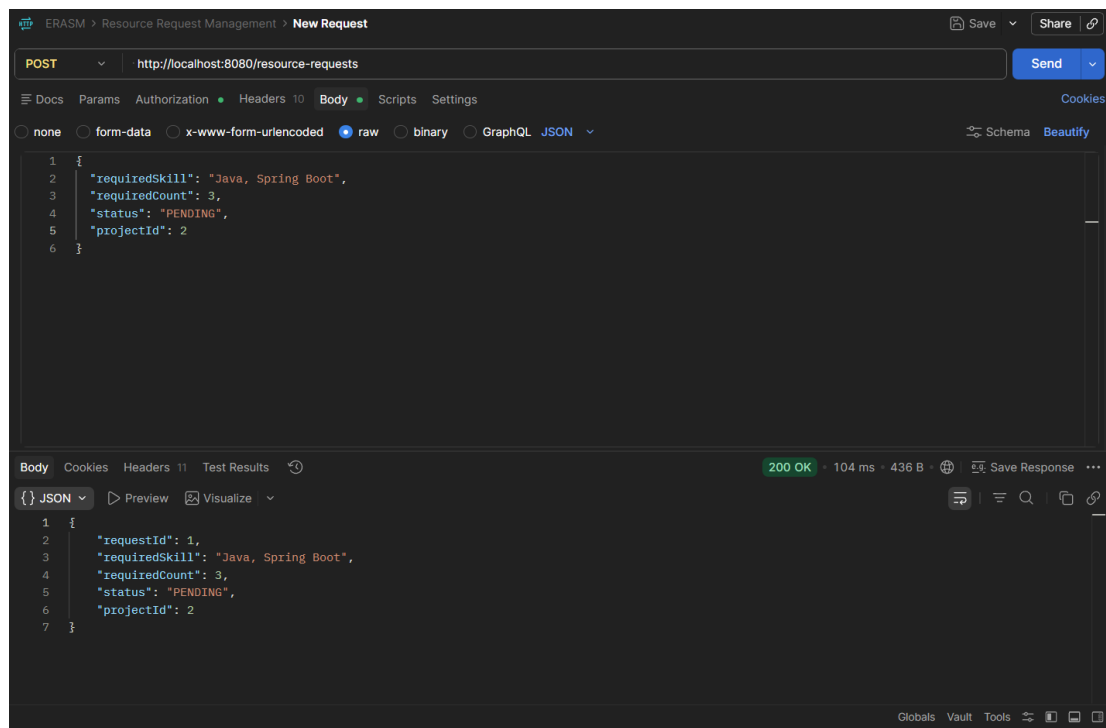


Figure 20: Submitting a new resource request via `POST /resource-requests`, created in `PENDING` status.

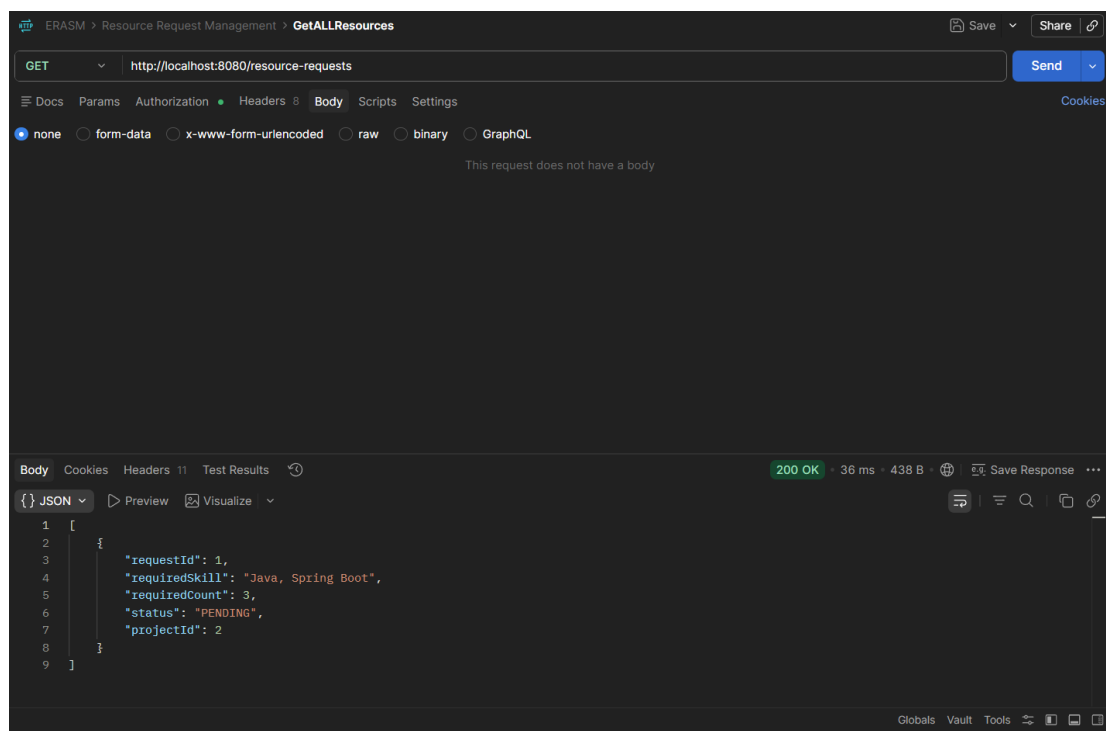


Figure 21: Retrieving all resource requests via `GET /resource-requests`.

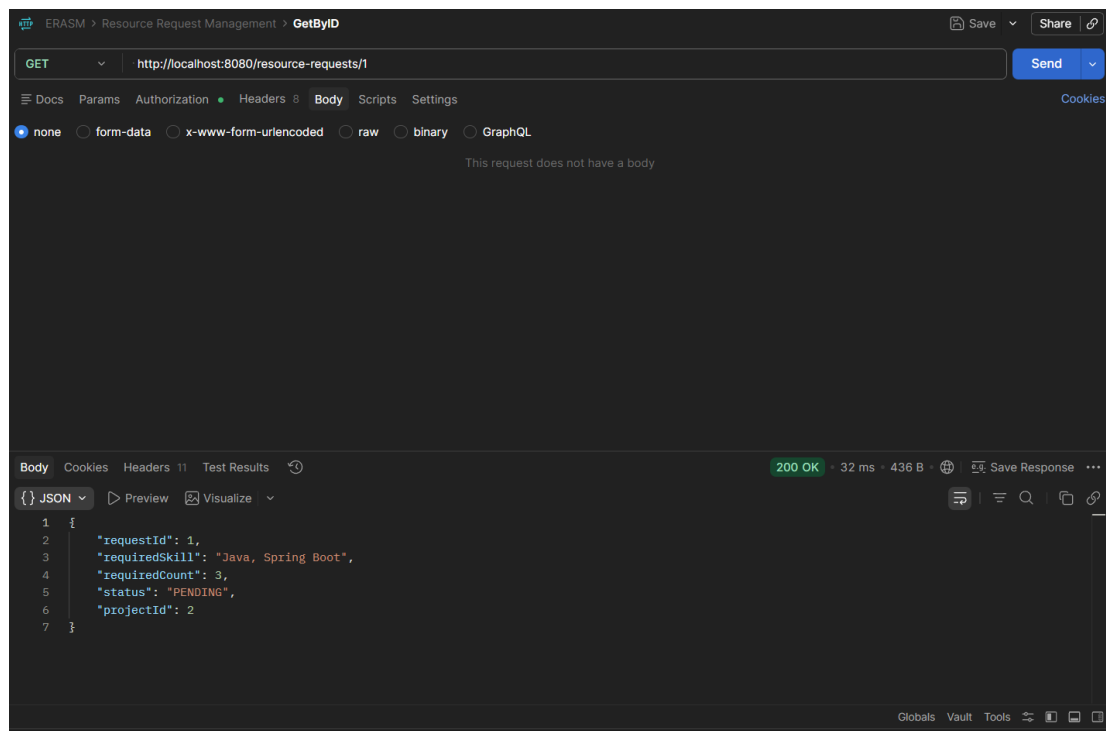


Figure 22: Retrieving a single resource request via GET /resource-requests/1.

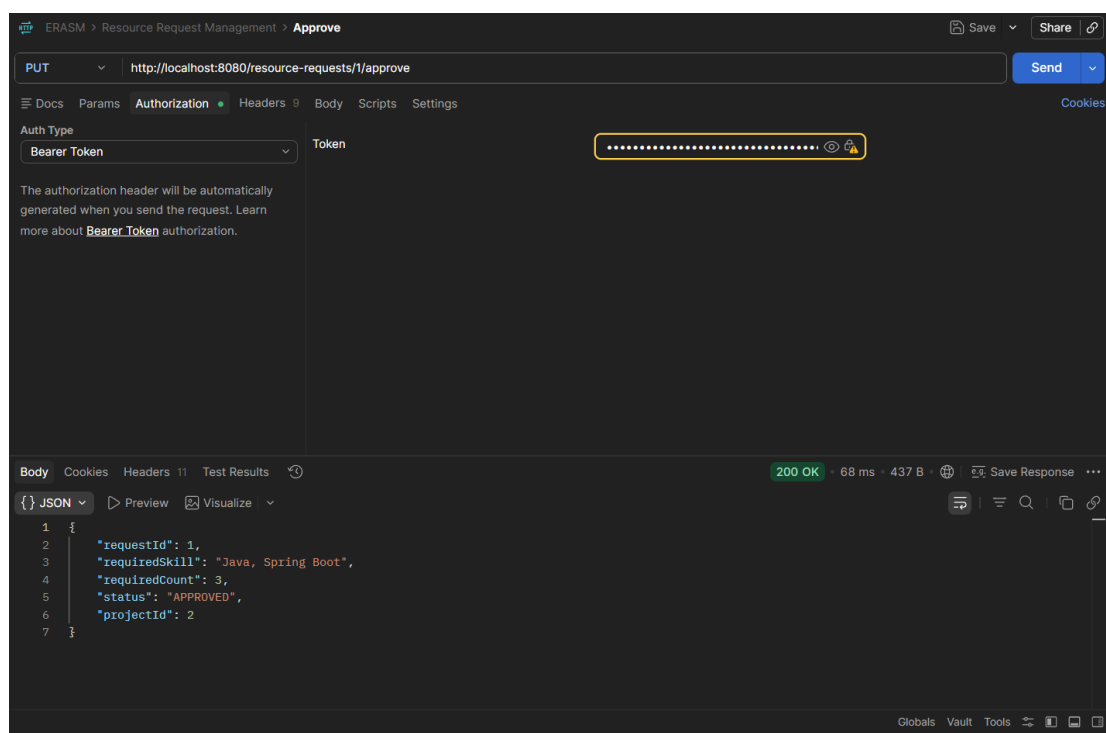


Figure 23: Approving a resource request via PUT /resource-requests/1/approve, transitioning status to APPROVED.

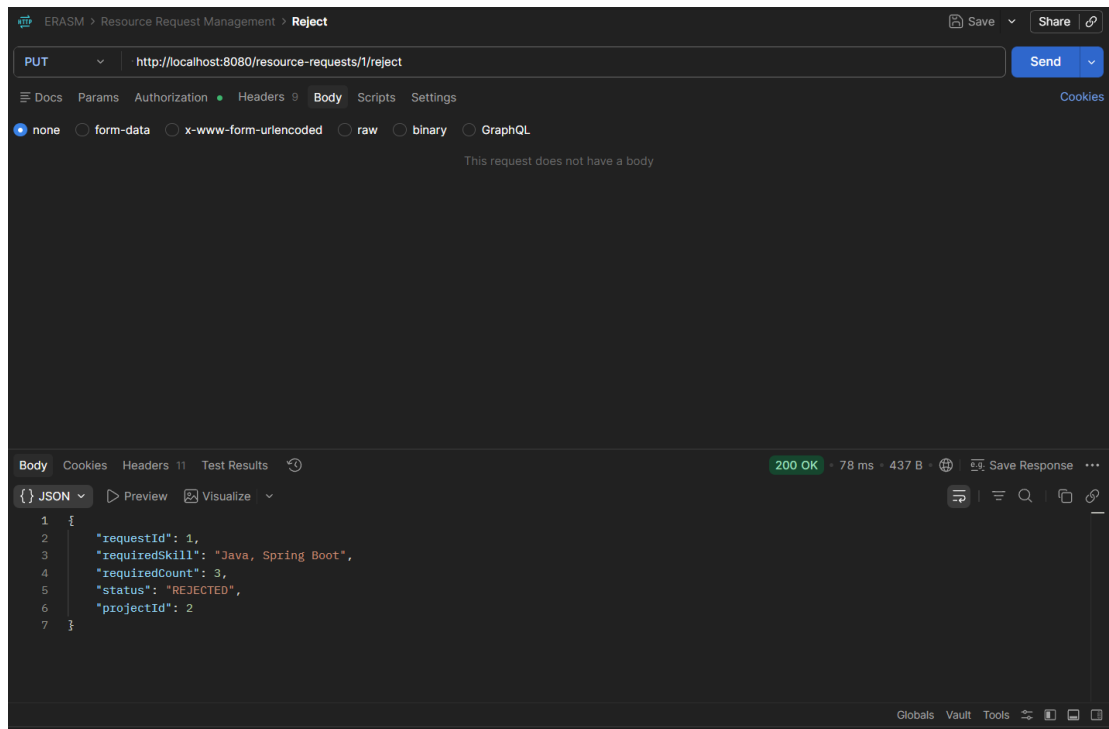


Figure 24: Rejecting a resource request via PUT /resource-requests/1/reject, transitioning status to REJECTED.

A.7 Audit Log

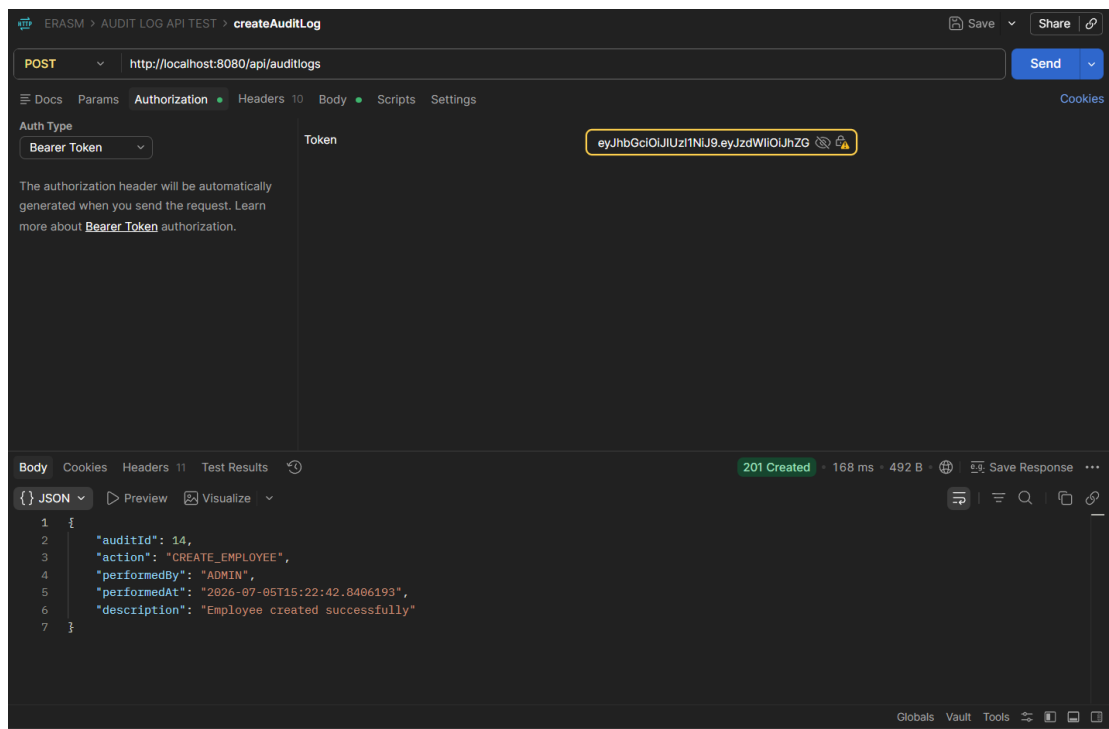


Figure 25: Creating an audit log entry via POST /api/auditlogs, returning 201 Created.

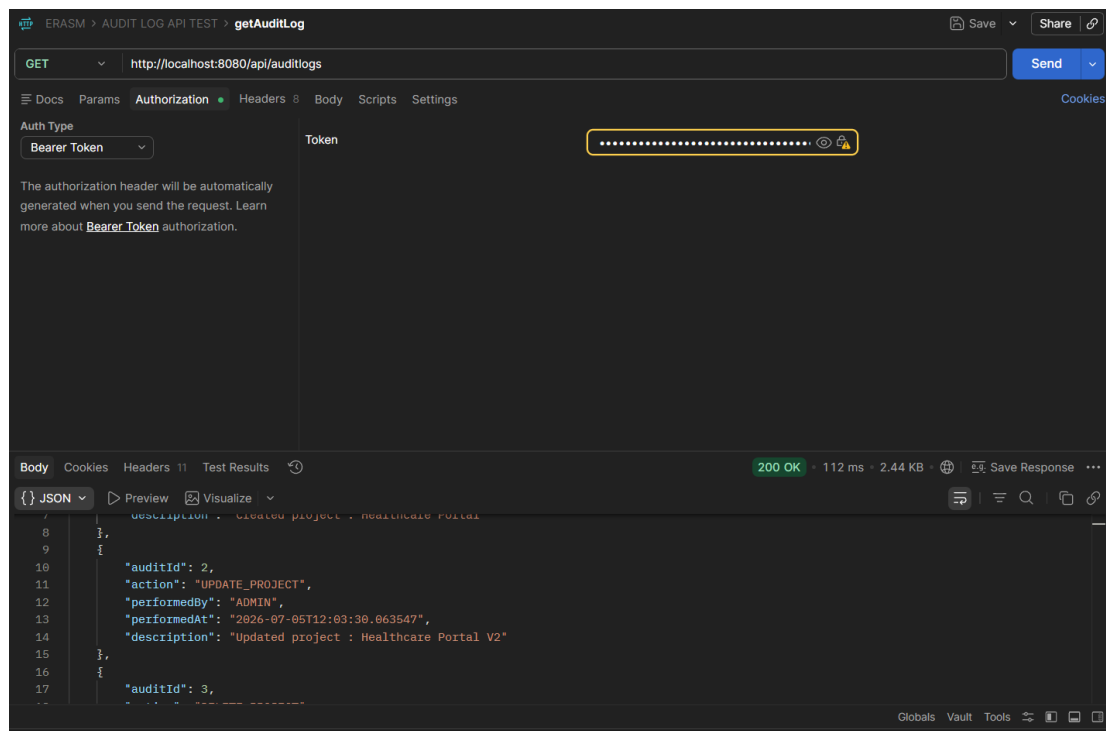


Figure 26: Retrieving all audit log entries via GET `/api/auditlogs`, showing recorded actions with actor and timestamp.

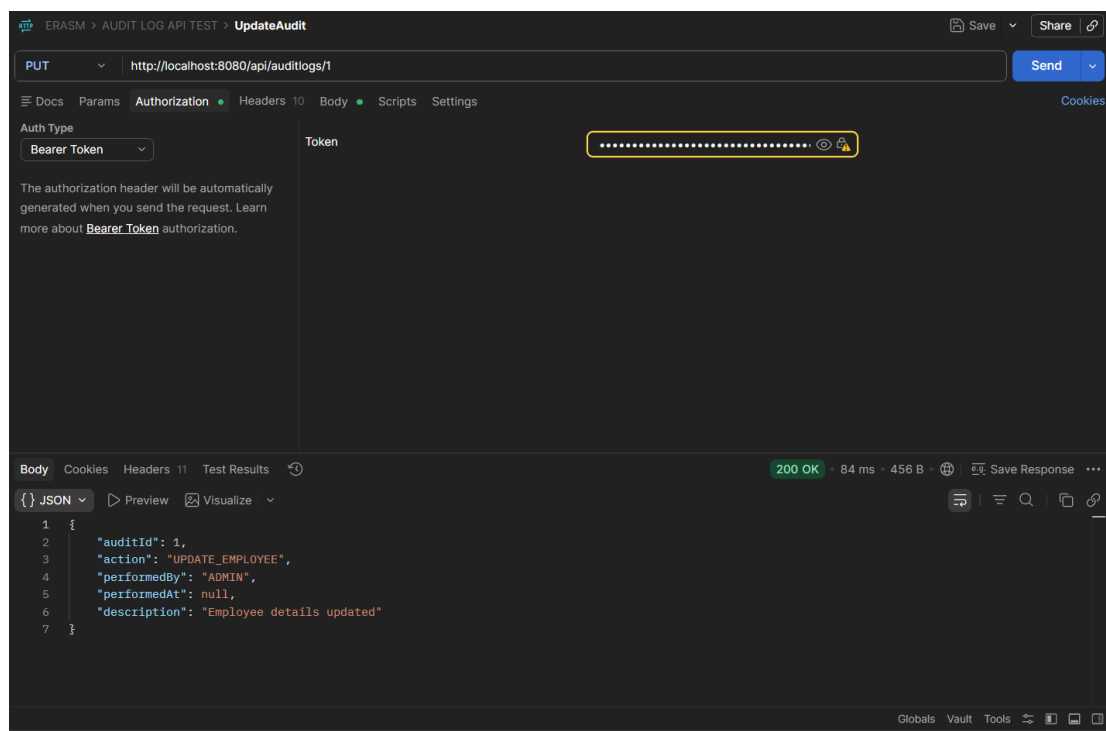


Figure 27: Updating an audit log entry via PUT `/api/auditlogs/1`.

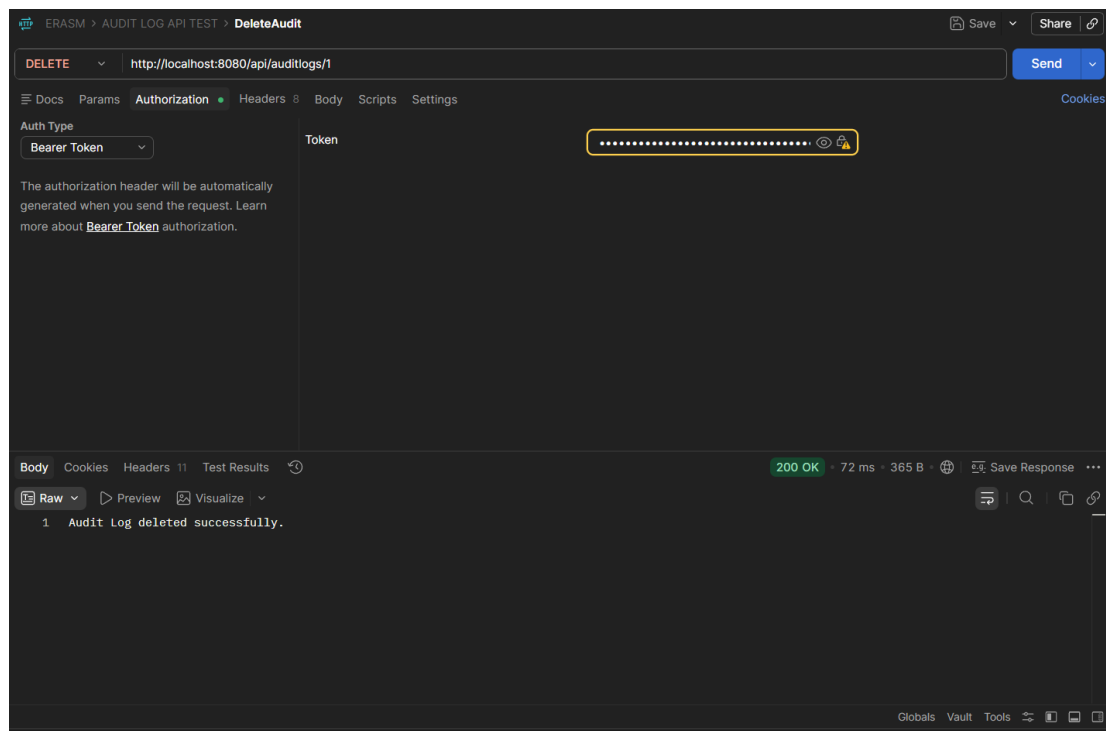


Figure 28: Deleting an audit log entry via DELETE /api/auditlogs/1, returning “Audit Log deleted successfully.”

B Entity-Relationship Diagram

The diagram below summarizes the persistent data model underlying the ERASM backend, as implemented through the JPA entity classes in the `com.erasm.entity` package and their corresponding MySQL tables. It uses standard crow's-foot notation and is scoped strictly to the ten tables that exist in the current codebase; it does not include speculative HR-suite tables (such as departments, attendance, leave, payroll, or performance reviews) that are not part of the present schema. It is provided here to support the database persistence testing described in Section 4.6 and Section 5, by making explicit the primary keys, foreign keys, and cardinalities that the test cases in this report rely on.

- Each **user** belongs to exactly one **role** (many-to-one), which drives RBAC enforcement.
- Each **employee** is linked to exactly one **user** account (one-to-one).
- **employee** and **skill** share a many-to-many relationship, resolved through the **employee_skills** join table, which additionally records skill level and years of experience.
- **employee** and **project** share a many-to-many relationship, resolved through the **allocations** join table, which records allocation percentage, dates, and billable/total hours.
- Each **project** may have multiple **resource_requests** (one-to-many), representing open staffing needs.
- Each **employee** may hold multiple **certifications** (one-to-many).
- **audit_logs** is maintained independently of the other tables, recording the action, actor, and timestamp of significant operations for traceability; it does not carry a formal foreign-key relationship in the current schema.

ERASM DATABASE ER DIAGRAM

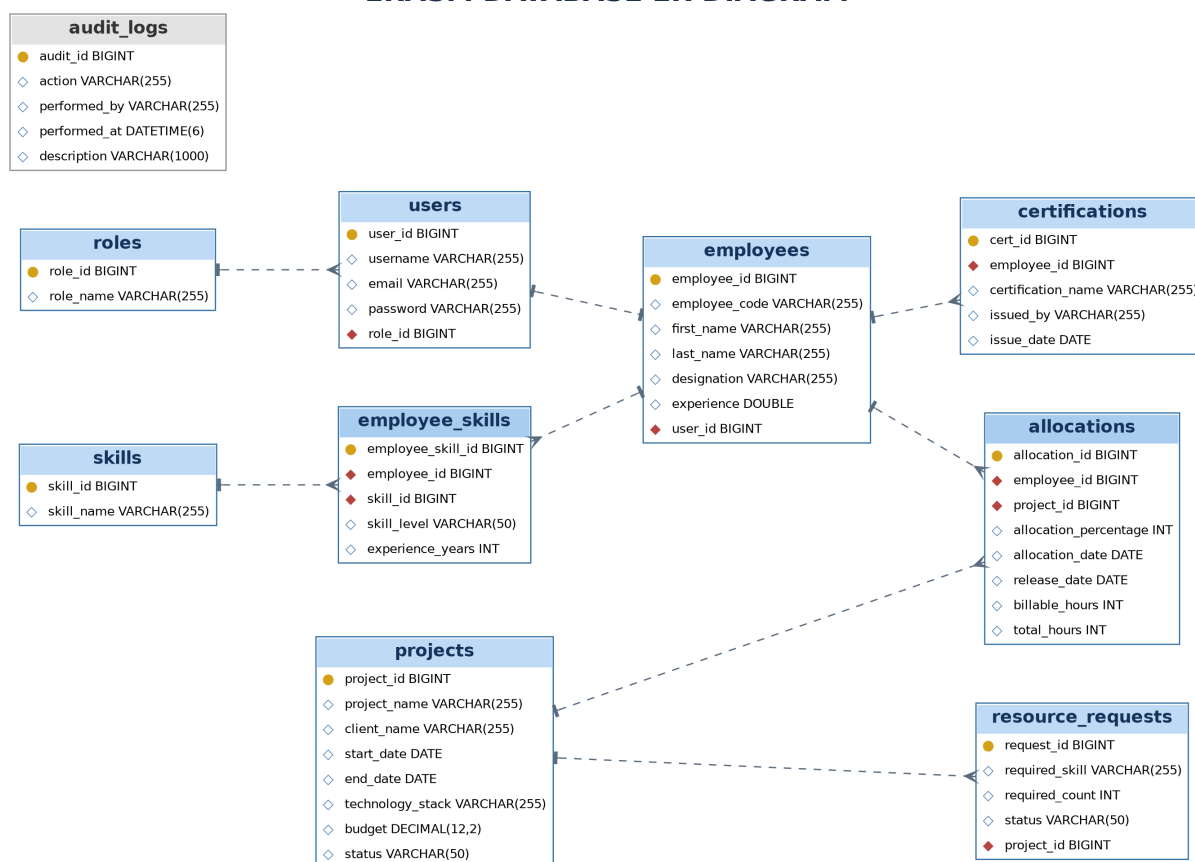


Figure 29: Entity-Relationship diagram of the ERASM database schema, derived from the JPA entity definitions.